Java - Unit 6

File Handling, GUI, and Java 8 Features

Syllabus

File Handling: File Processing, Primitive Data Processing, Object Data Processing, Connecting Java with database (JDBC/ODBC), Java GUI: Swing, Components, Layout Manager: Flow, Border, Grid and Card, Label, Button, Choice, List, Event Handling (mouse, key)

File Handling in Java

File Handling in Java:

- File handling is an important part of any application.
- The File class from the java.io package, allows us to work with files.
- To use the File class, create an object of the class, and specify the filename or directory name.

Why file handling is important?

- Programs store data temporarily in RAM → data lost when program ends
- Files provide permanent storage
- Examples:
  - Saving student records
  - Reading configuration files
  - Writing logs

File Handling in Java

```
import java.io.File;  // Import the File class

File myObj = new File("filename.txt"); // Specify the filename
```

1. Java File class represents the files and directory pathnames in an abstract manner.
2. This class is used for creation of files and directories, file searching, file deletion, etc.
3. The File object represents the actual file/directory on the disk.

File Handling in Java

Method	Type	Description
`canRead()`	Boolean	Tests whether the file is readable or not
`canWrite()`	Boolean	Tests whether the file is writable or not
`createNewFile()`	Boolean	Creates an empty file
`delete()`	Boolean	Deletes a file
`exists()`	Boolean	Tests whether the file exists
`getName()`	String	Returns the name of the file
`getAbsolutePath()`	String	Returns the absolute pathname of the file
`length()`	Long	Returns the size of the file in bytes
`list()`	String[]	Returns an array of the files in the directory
`mkdir()`	Boolean	Creates a directory

File Handling in Java

Types of Files:

Type	Description	Example Classes
Text File	Human-readable, plain text	FileReader, FileWriter
Binary File	Stores data in binary form	DataInputStream, DataOutputStream

File Handling in Java

Java Create and Write to File:

- To create a file in Java, you can use the createNewFile() method.
- This method returns a boolean value: true if the file was successfully created, and false if the file already exists.
- Note that the method is enclosed in a try...catch block.
- This is necessary because it throws an IOException if an error occurs (if the file cannot be created for some reason):

File Handling in Java

Java File Operation Methods

Operation	Method	Package
To create file	`createNewFile()`	`java.io.File`
To read file	`read()`	`java.io.FileReader`
To write file	`write()`	`java.io.FileWriter`
To delete file	`delete()`	`java.io.File`

File Handling in Java

Creating a file

```
import java.io.File;  // Import the File class
import java.io.IOException;  // Import the IOException class to handle errors

public class CreateFile {
  public static void main(String[] args) {
    try {
      File myObj = new File("filename.txt");
      if (myObj.createNewFile()) {
        System.out.println("File created: " + myObj.getName());
      } else {
        System.out.println("File already exists.");
      }
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

File created: filename.txt

File Handling in Java

Writing File to a Specific Directory

■ To create a file in a specific directory (requires permission), specify the path of the file and use double backslashes to escape the "\" character (for Windows).
■ On Mac and Linux you can just write the path, like: /Users/name/filename.txt

```
File myObj = new File("C:\\Users\\MyName\\filename.txt");
```

■ In the following example, we use the FileWriter class together with its write() method to write some text to the file we created in the example above. ■ Note that when you are done writing to the file, you should close it with the close() method:

File Handling in Java

```java
import java.io.FileWriter;   // Import the FileWriter class
import java.io.IOException;  // Import the IOException class to handle errors

public class WriteToFile {
  public static void main(String[] args) {
    try {
      FileWriter myWriter = new FileWriter("filename.txt");
      myWriter.write("Files in Java might be tricky, but it is fun enough!");
      myWriter.close();
      System.out.println("Successfully wrote to the file.");
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

```
Successfully wrote to the file.
```

File Handling in Java

Read File

In the following example, we use the Scanner class to read the contents of the text file.

```
import java.io.File;  // Import the File class
import java.io.FileNotFoundException;  // Import this class to handle errors
import java.util.Scanner; // Import the Scanner class to read text files

public class ReadFile {
  public static void main(String[] args) {
    try {
      File myObj = new File("filename.txt");
      Scanner myReader = new Scanner(myObj);
      while (myReader.hasNextLine()) {
        String data = myReader.nextLine();
        System.out.println(data);
      }
      myReader.close();
    } catch (FileNotFoundException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
```

```
Files in Java might be tricky, but it is fun enough!
```

Deleting a File

```java
import java.io.File;  // Import the File class

public class DeleteFile {
  public static void main(String[] args) {
    File myObj = new File("filename.txt");
    if (myObj.delete()) {
      System.out.println("Deleted the file: " + myObj.getName());
    } else {
      System.out.println("Failed to delete the file.");
    }
  }
}
```

```
Deleted the file: filename.txt
```

File Input Stream

- A FileInputStream obtains input bytes from a file in a file system.
- What files are available depends on the host environment.
- FileInputStream is meant for reading streams of raw bytes such as image data.
- For reading streams of characters, consider using FileReader.

Counting Number of Characters in a file

```java
// Java program to count the
// number of charaters in a file
import java.io.*;

public class Test
{
    public static void main(String[] args) throws IOException
    {
        File file = new File("C:\\Users\\Mayank\\Desktop\\1.txt");
        FileInputStream fileStream = new FileInputStream(file);
        InputStreamReader input = new InputStreamReader(fileStream);
        BufferedReader reader = new BufferedReader(input);

        String line;

        // Initializing counters
        int countWord = 0;
        int sentenceCount = 0;
        int characterCount = 0;
        int paragraphCount = 1;
        int whitespaceCount = 0;
```

cont……

```
// Reading line by line from the
// file until a null is returned
while((line = reader.readLine()) != null)
{
    if(line.equals(""))
    {
        paragraphCount++;
    } else {
        characterCount += line.length();

        // \\s+ is the space delimiter in java
        String[] wordList = line.split("\\s+");

        countWord += wordList.length;
        whitespaceCount += countWord -1;

        // [!?.:]+ is the sentence delimiter in java
        String[] sentenceList = line.split("[!?.:]+");

        sentenceCount += sentenceList.length;
    }
}
```

cont…

```
        System.out.println("Total word count = " + countWord);
        System.out.println("Total number of sentences = " + sentenceCount);
        System.out.println("Total number of characters = " + characterCount);
        System.out.println("Number of paragraphs = " + paragraphCount);
        System.out.println("Total number of whitespaces = " + whitespaceCount);
    }
}
```

Output

```
Total word count = 5
Total number of sentences = 3
Total number of characters = 21
Number of paragraphs = 2
Total number of whitespaces = 7
```

Primitive Data Types

- In Java, the primitive data types are the predefined data types of Java.
- They specify the size and type of any standard values.
- Java has 8 primitive data types namely byte, short, int, long, float, double, char and boolean.
- When a primitive data type is stored, it is the stack that the values will be assigned.
- When a variable is copied then another copy of the variable is created and changes made to the copied variable will not reflect changes in the original variable.

Primitive Data Types

Integer: This group includes byte, short, int, long

- byte : It is 1 byte(8-bits) integer data type. Value range from -128 to 127. Default value zero. example: byte b=10;
- short : It is 2 bytes(16-bits) integer data type. Value range from -32768 to 32767. Default value zero. example: short s=11;
- int : It is 4 bytes(32-bits) integer data type. Value range from -2147483648 to 2147483647. Default value zero. example: int i=10;
- long : It is 8 bytes(64-bits) integer data type. Value range from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. Default value zero. example: long l=100012;

Primitive Data Types

```
public class Demo {

   public static void main(String[] args)

   {

      // byte type

      byte b = 20;

      System.out.println("b= " + b);

      // short type

      short s = 20;

      System.out.println("s= " + s);

      // int type

      int i = 20;

      System.out.println("i= " + i);

      // long type

      long l = 20;

      System.out.println("l= " + l);    }
```

Output:
b= 20
s= 20
i= 20
l= 20

Primitive Data Types

Floating-Point Number:

- This group includes float, double
- float : It is 4 bytes(32-bits) float data type. Default value 0.0f.

  example: float ff=10.3f;
- double: It is 8 bytes(64-bits) float data type. Default value 0.0d.

  example: double db=11.123;

Primitive Data Types

```
public class Demo {

    public static void main(String[] args)

    {

        // float type

        float f = 20.25f;

        System.out.println("f= " + f);

        // double type

        double d = 20.25;

        System.out.println("d= " + d);

    }

}
```

Output

```
f= 20.25
d= 20.25
```

Primitive Data Types

Characters: This group represent char, which represent symbols in a character set, like letters and numbers.

char: It is 2 bytes(16-bits) unsigned unicode character. Range 0 to 65,535.

example: char c='a';

```
public class Demo {
 public static void main(String[] args) {
        char ch = 'S';
        System.out.println(ch);
        char ch2 = '&';
        System.out.println(ch2);
        char ch3 = '$';
        System.out.println(ch3);
    }
}
```

Output:

```
S
&
$
```

Primitive Data Types

Boolean: Boolean type is used when we want to test a particular condition during the execution of the program. There are only two values that a Boolean type can take: true or false.

Remember, both these words have been declared as keyword. Boolean type is denoted by the keyword Boolean and uses only 1 bit of storage.

```
public class Demo {
   public static void main(String[] args) {
      boolean t = true;
      System.out.println(t);
      boolean f = false;
      System.out.println(f);

   }}
```

Output:

```
true
false
```

Object Data Types

These are also referred to as Non-primitive or Reference Data Type. They are so-called because they refer to any particular object. Unlike the primitive data types, the non-primitive ones are created by the users in Java. Examples include arrays, strings, classes, interfaces etc.

```
import java.lang.*;
import java.util.*;

class GeeksForGeeks {
    public static void main(String ar[])
    {
        System.out.println("PRIMITIVE DATA TYPES\n");
        int x = 10;
        int y = x;
        System.out.print("Initially: ");
        System.out.println("x = " + x + ", y = " + y);

        // Here the change in the value of y
        // will not affect the value of x
        y = 30;

        System.out.print("After changing y to 30: ");
        System.out.println("x = " + x + ", y = " + y);
        System.out.println(
            "**Only value of y is affected here "
            + "because of Primitive Data Type\n");

        System.out.println("REFERENCE DATA TYPES\n");
        int[] c = { 10, 20, 30, 40 };
```

```
        // Here complete reference of c is copied to d
        // and both point to same memory in Heap
        int[] d = c;

        System.out.println("Initially");
        System.out.println("Array c: "
                           + Arrays.toString(c));
        System.out.println("Array d: "
                           + Arrays.toString(d));

        // Modifying the value at
        // index 1 to 50 in array d
        System.out.println("\nModifying the value at "
                           + "index 1 to 50 in array d\n");
        d[1] = 50;

        System.out.println("After modification");
        System.out.println("Array c: "
                           + Arrays.toString(c));
        System.out.println("Array d: "
                           + Arrays.toString(d));
        System.out.println(
            "**Here value of c[1] is also affected "
            + "because of Reference Data Type\n");
    }
}
```

Output:

PRIMITIVE DATA TYPES

Initially: x = 10, y = 10
After changing y to 30: x = 10, y = 30
**Only value of y is affected here because of Primitive Data Type

REFERENCE DATA TYPES

Initially
Array c: [10, 20, 30, 40]
Array d: [10, 20, 30, 40]

Modifying the value at index 1 to 50 in array d

After modification
Array c: [10, 50, 30, 40]
Array d: [10, 50, 30, 40]
**Here value of c[1] is also affected because of Reference Data Type

Graphical User Interfaces

Java's AWT and Swing APIs

AWT and Swing

- Java provides two sets of components for GUI programming:
  - AWT: classes in the **`java.awt`** package
  - Swing: classes in the **`javax.swing`** package

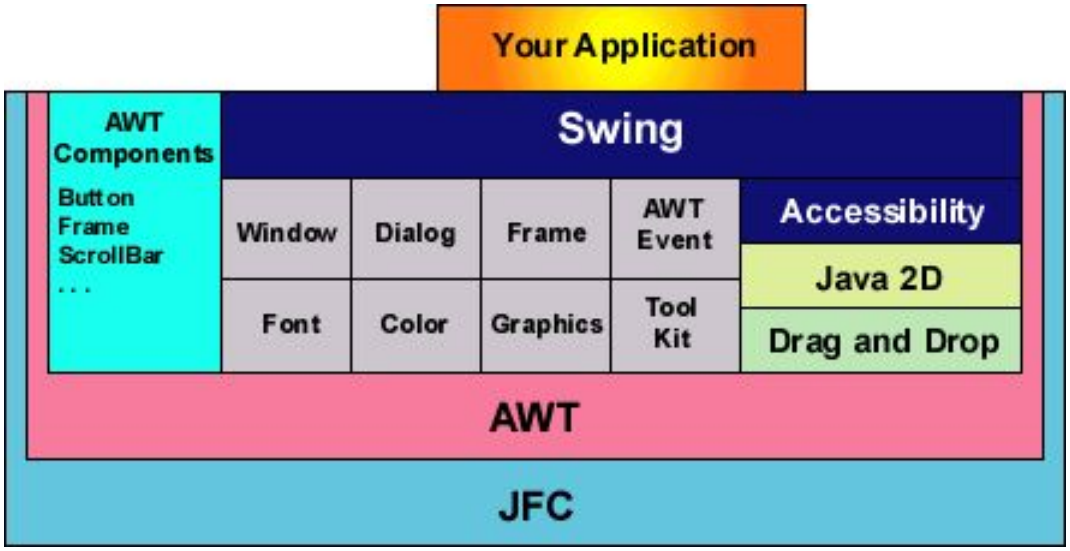
Abstract Window Toolkit (AWT)

- The Abstract Window Toolkit is a portable GUI library.
- AWT provides the connection between your application and the native GUI.
- AWT provides a high-level abstraction since it hides you from the underlying details of the GUI your program will be running on.
- AWT components depend on native code counterparts (called peers) to handle their functionality. Thus, these components are often called ***heavyweight*** components.

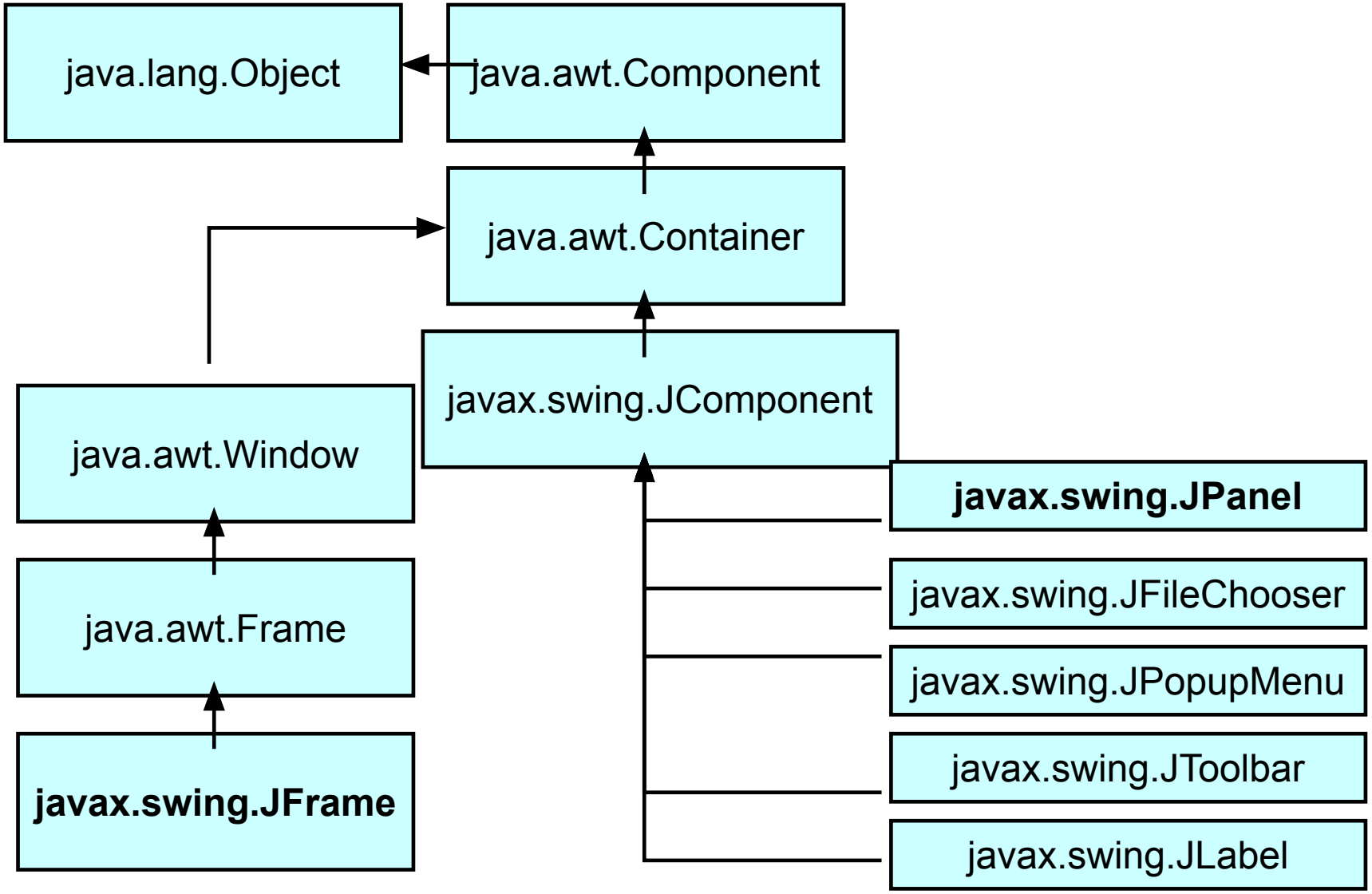
Swing

- Swing implements GUI components that build on AWT technology.
- Swing is implemented entirely in Java.
- Swing components do not depend on peers to handle their functionality. Thus, these components are often called ***lightweight*** components.

Swing Stack


Your Application
AWT Components
Button
Frame
ScrollBar
. . .
Swing
Window
Dialog
Frame
AWT Event
Accessibility
Java 2D
Font
Color
Graphics
Tool Kit
Drag and Drop
AWT
JFC

Some AWT and Swing Classes



Graphical User Interfaces

- A Graphical User Interface (GUI) is created with at least three kinds of objects
  - components
  - events
  - Listeners
- Java standard class library provides these objects for us

Components and Containers

- A GUI *component* defines a screen element to display information or allow the user to interact with the program
  - buttons, text fields, labels, menus, etc.
- A *container* is a special component that holds and organizes other components
  - dialog boxes, applets, frames, panels, etc.

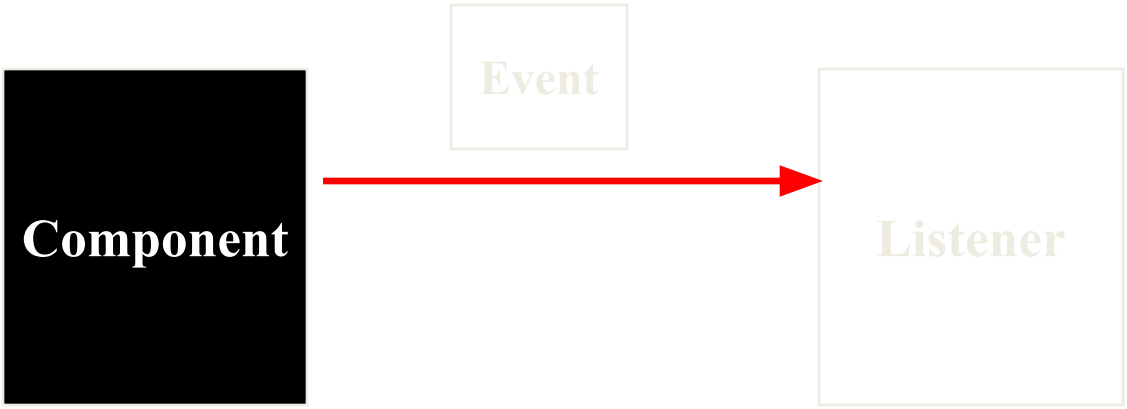
Events

- An *event* is an object that represents some activity to which we may want to respond
- For example, we may want our program to perform some action when the following occurs:
  - the mouse is moved
  - a mouse button is clicked
  - a graphical button is clicked
  - a keyboard key is pressed
- Events often correspond to user actions, but not always
- A program oriented around this type of interaction is called “event-driven”

Events and Listeners

- The Java standard class library contains several classes that represent typical events
- Components, such as an applet or a graphical button, generate (fire) an event when it occurs
- Other objects, called *listeners*, wait for events to occur
- We can write listener objects to do whatever we want when an event occurs
- A listener object is often defined using an inner class

Events and Listeners



This object may generate an event

This object waits for and responds to an event

When an event occurs, the generator calls the appropriate method of the listener, passing an object that describes the event

Main Steps in GUI Programming

To make any graphic program work we must be able to create windows and add content to them.

To make this happen we must:

1. Import the `awt` or `swing` packages.
2. Set up a top-level container.
3. Fill the container with GUI components.
4. Install listeners for GUI Components.
5. Display the container.

Hello World Example

```
import javax.swing.*;
public class HelloWorldSwing {
   public static void main(String[] args) {
    JFrame frame = new JFrame("HelloWorldSwing");
    final JLabel label = new JLabel("Hello World");
    frame.getContentPane().add(label);
    frame.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE);
    frame.pack();
    frame.setVisible(true);
   }
}
```

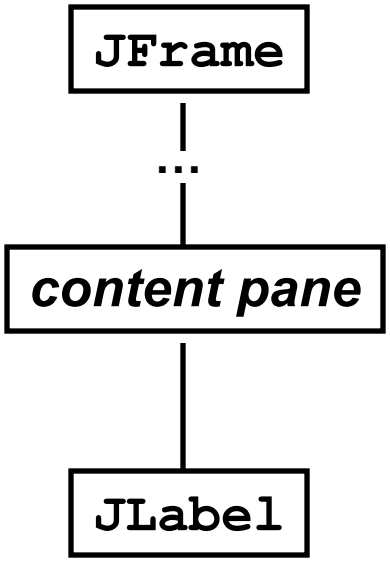
Top-level Containers

- There are three top-level Swing containers
  - **`JFrame`**: window that has decorations, such as a border, a title, and buttons for iconifying and closing the window
  - **`JDialog`**: a window that's dependent on another window
  - **`JApplet`**: applet's display area within a browser window

Containment Hierarchy

- In the Hello World example, there was a *content pane*.
- Every top-level container indirectly contains an intermediate container known as a *content pane*.
- As a rule, the content pane contains, directly or indirectly, all of the visible components in the window's GUI.
- To add a component to a container, you use one of the various forms of the **`add`** method.

Containment Hierarchy of the Hello World Example



Event Example

```
public class SwingApplication extends JFrame {
  private static String labelPrefix = "Number of    button
   clicks: ";
  private int numClicks = 0;
  JLabel label = new JLabel(labelPrefix + "0    ");
  public SwingApplication(String title) {
    super(title);
    JButton button = new JButton("I'm a Swing       button!");
    button.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent e) {
        label.setText(labelPrefix + ++numClicks);
      }
    });
    JPanel panel = new JPanel();
    panel.add(button);
    panel.add(label);
    getContentPane().add(panel);
    pack();
    setVisible(true);
  }
  public static void main(String[] args) {
    new SwingApplication("SwingApplication");  }}
```

Handling Events

- Every time the user types a character or pushes a mouse button, an event occurs.
- Any object can be notified of the event.
- All the object has to do is implement the appropriate interface and be registered as an event listener on the appropriate event source.

How to Implement an Event Handler

- Every event handler requires three pieces of code:
  1. declaration of the event handler class that implements a listener interface or extends a class that implements a listener interface

     ```
     public class MyClass implements ActionListener {
     ```

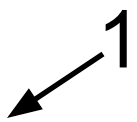
  2. registration of an instance of the event handler class as a listener

     ```
     someComponent.addActionListener(instanceOfMyClass);
     ```

  3. providing code that implements the methods in the listener interface in the event handler class

     ```
     public void actionPerformed(ActionEvent e) {
     ...//code that reacts to the action...
     }
     ```

A Simpler Event Example




```
public class ButtonClickExample extends JFrame    implements
   ActionListener {
  JButton b = new JButton("Click me!");
  public ButtonClickExample() {
    b.addActionListener(this);
    getContentPane().add(b);
    pack();
    setVisible(true);
  }
  public void actionPerformed(ActionEvent e) {
    b.setBackground(Color.CYAN);
  }
  public static void main(String[] args) {
    new ButtonClickExample();
  }
}
```

Example Summary

- (1) declares a class that implements a listener interface (i.e. **`ActionListener`**)
- (2) registers an instance of this class with the event source
- (3) defines the action to take when the event occurs

JTextField Example (1)

```
public class CelsiusConverter implements ActionListener
   {
  JFrame converterFrame;
  JPanel converterPanel;
  JTextField tempCelsius;
  JLabel celsiusLabel, fahrenheitLabel;
  JButton convertTemp;
  public CelsiusConverter() {
    converterFrame = new JFrame("Convert Celsius to
      Fahrenheit");
    converterPanel = new JPanel();
    converterPanel.setLayout(new GridLayout(2, 2));
    addWidgets();
    converterFrame.getContentPane().add(converterPanel,
      BorderLayout.CENTER);
    converterFrame.setDefaultCloseOperation(
      JFrame.EXIT_ON_CLOSE);
    converterFrame.pack();
    converterFrame.setVisible(true);
  }
```

JTextField Example (2)

```
private void addWidgets() {
  tempCelsius = new JTextField(2);
  celsiusLabel = new JLabel("Celsius",
    SwingConstants.LEFT);
  convertTemp = new JButton("Convert...");
  fahrenheitLabel = new JLabel("Fahrenheit",
    SwingConstants.LEFT);
  convertTemp.addActionListener(this);
  converterPanel.add(tempCelsius);
  converterPanel.add(celsiusLabel);
  converterPanel.add(convertTemp);
  converterPanel.add(fahrenheitLabel);
}
```

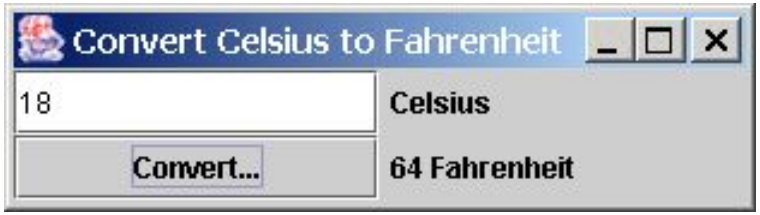
JTextField Example (3)

```
  public void actionPerformed(ActionEvent event) {
    int tempFahr = (int)((Double.parseDouble(
      tempCelsius.getText())) * 1.8 + 32);
    fahrenheitLabel.setText(tempFahr + " Fahrenheit");
  }

  public static void main(String[] args) {
    try {
      UIManager.setLookAndFeel(

   UIManager.getCrossPlatformLookAndFeelClassName());
    } catch(Exception e) {}

    CelsiusConverter converter = new CelsiusConverter();
  }
} // end CelciusConverter class
```



Dialogs - `JOptionPane`

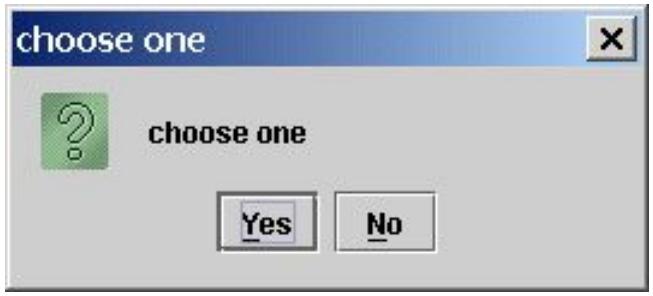
- Dialogs are windows that are more limited than frames.
- Every dialog is dependent on a frame. When that frame is destroyed, so are its dependent dialogs. When the frame is iconified, its dependent dialogs disappear from the screen. When the frame is deiconified, its dependent dialogs return to the screen.
- To create simple dialogs, use the **`JOptionPane`** class.
- The dialogs that **`JOptionPane`** provides are *modal*.
- When a modal dialog is visible, it blocks user input to all other windows in the program.

JOptionPane Examples

```
// show an error dialog
JOptionPane.showMessageDialog(null, "alert", "alert",
   JOptionPane.ERROR_MESSAGE);
```

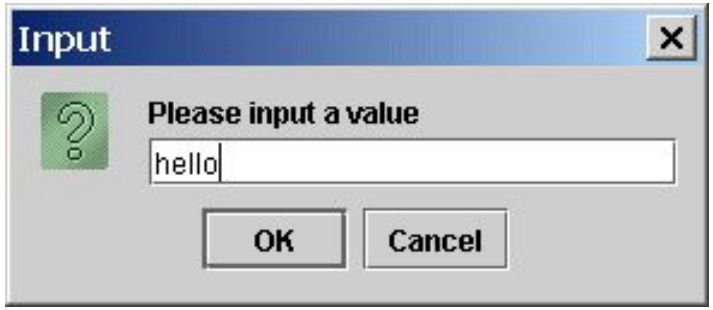
JOptionPane Examples

```
// show Yes/No dialog
int x = JOptionPane.showConfirmDialog(null,
   "choose one", "choose one", JOptionPane.YES_NO_OPTION);
System.out.println("User clicked button " + x);
```



JOptionPane Examples

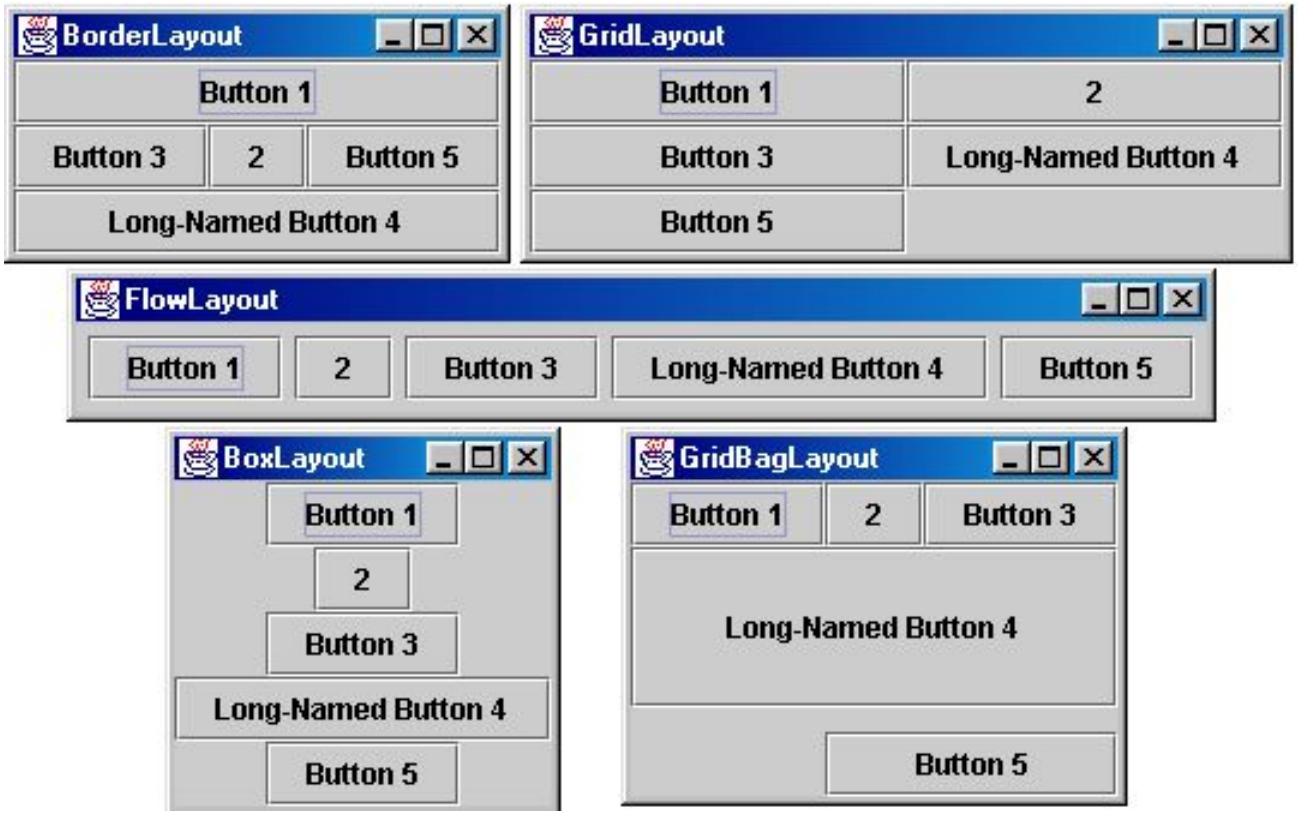
```
// show input dialog
String inputValue = JOptionPane.showInputDialog("Please input
   " +
   "a value");
   System.out.println("User entered " + inputValue);
```



Layout Management

- Layout managers control the size and arrangement of components in a container.
- There are 6 common layout managers:
  - **`BorderLayout`**
  - **`BoxLayout`**
  - **`FlowLayout`**
  - **`GridBagLayout`**
  - **`GridLayout`**
  - **`CardLayout`**

Layout Management



`FlowLayout`

- Components are placed in a row from left to right in the order in which they are added.
- A new row is started when no more components can fit in the current row.
- The components are centered in each row by default.
- The programmer can specify the size of both the vertical and horizontal gaps between the components.
- **`FlowLayout`** is the default layout for **`JPanel`**s.

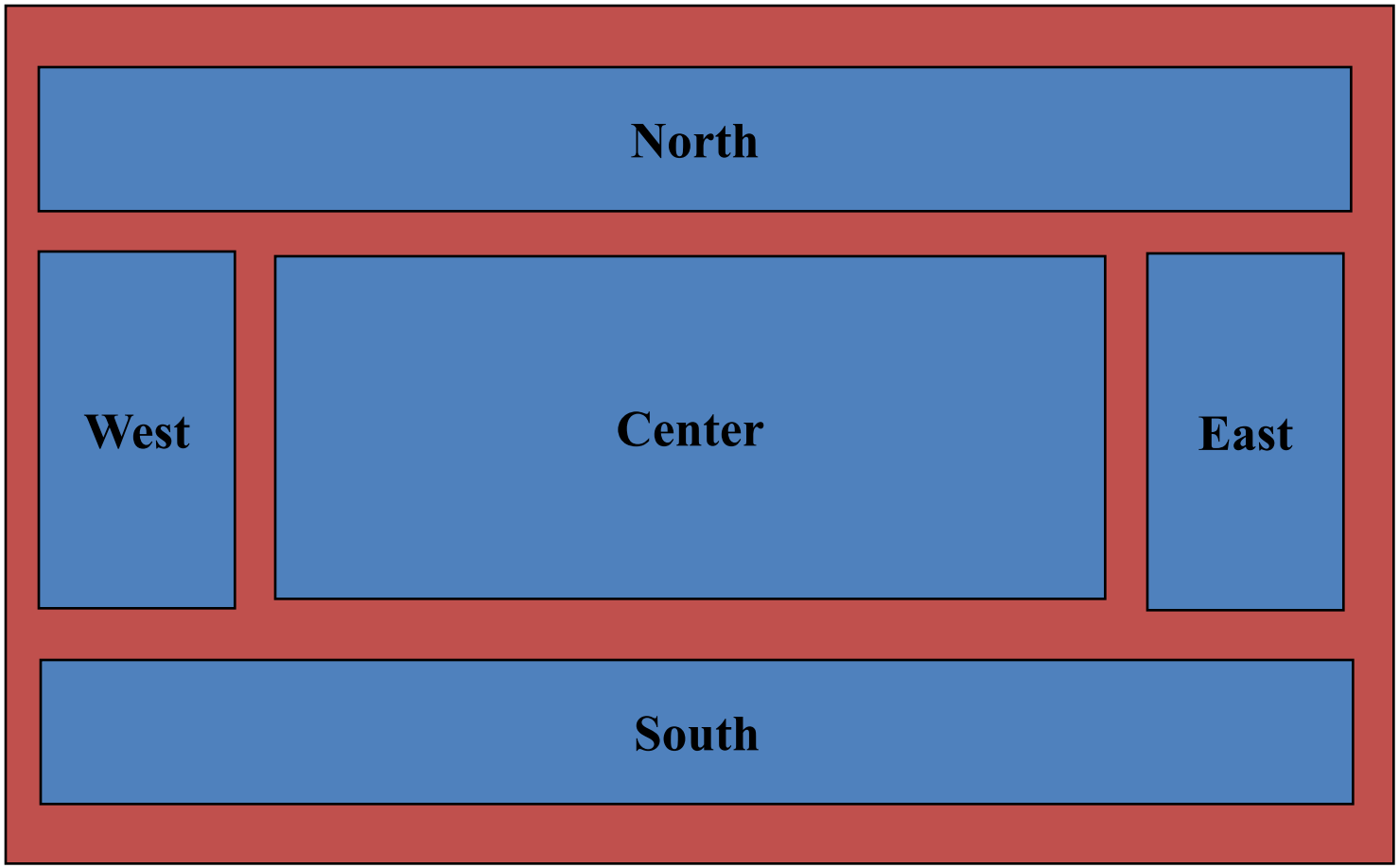
FlowLayout Example

```
public class FlowLayoutTest extends JFrame {
   JButton b1=new JButton("Red"),
   b2=new JButton("Green"),b3=new JButton("Blue"),
   b4=new JButton("Yellow"),b5=newJButton("Pink");
    public FlowLayoutTest() {
       setTitle("FlowLayout Test");
       Container pane = getContentPane();
       pane.setLayout(new FlowLayout());
       setBounds(0,0,400,100);
       pane.add(b1); pane.add(b2); pane.add(b3);
       pane.add(b4); pane.add(b5);
    }
    public static void main(String args[]) {
       JFrame f = new FlowLayoutTest();
       f.setVisible(true);
    }
}
```

BorderLayout

- Defines five locations where a component or components can be added:
  - North, South, East, West, and Center
- The programmer specifies the area in which a component should appear.
- The relative dimensions of the areas are governed by the size of the components added to them.

BorderLayout



Border-Layout Example


```
public class BorderLayoutTest extends JFrame {
   JButton b1=new JButton("Red"),
   b2=new JButton("Green"),b3=new JButton("Blue"),
   b4=new JButton("Yellow"),b5=new JButton("Pink");
   public BorderLayoutTest() {
    setTitle("BorderLayout Test");
    Container pane = getContentPane();
    pane.setLayout(new BorderLayout());
    setBounds(0,0,400,150);
    pane.add(b1,"North"); pane.add(b2,"South");
   pane.add(b3,"East");
        pane.add(b4,"West"); pane.add(b5,"Center");
   }
   public static void main(String args[]) {
    JFrame f = new BorderLayoutTest();
        f.setVisible(true);
   }
}
```

note extra parameter

GridLayout

- Components are placed in a grid with a user-specified number of columns and rows.
- Each component occupies exactly one grid cell.
- Grid cells are filled left to right and top to bottom.
- All cells in the grid are the same size.
- Specifying zero for either rows or columns means any number of items can be placed in that row or column.

GridLayout Example


```
public class GridLayoutTest extends JFrame {
   JButton b1=new JButton("Red"),
   b2=new JButton("Green"),b3=new JButton("Blue"),
   b4=new JButton("Yellow"),b5=new JButton("Pink");
    public GridLayoutTest() {
      setTitle("GridLayout Test");
      Container pane = getContentPane();
      pane.setLayout(new GridLayout(2,3));
      setBounds(0,0,300,100);
     pane.add(b1); pane.add(b2); pane.add(b3);
     pane.add(b4); pane.add(b5);
    }
    public static void main(String args[]) {
       JFrame f = new GridLayoutTest();
       f.setVisible(true);
    }
}
```

`CardLayout`

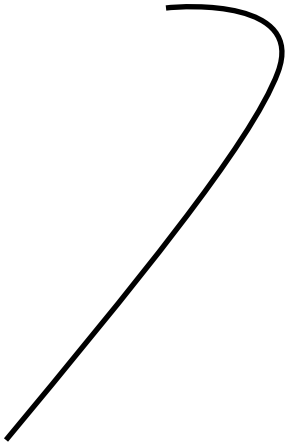
- Components governed by a card layout are "stacked" such that only one component is displayed on the screen at any one time.
- Components are ordered according to the order in which they were added to the container.
- Methods control which component is currently visible in the container.
- **`CardLayout`**s might be appropriate for wizards (with the Next >> buttons).

`CardLayout` Example (1 of 3)

```
public class CardLayoutTest extends JFrame
    implements ActionListener {
  JButton b1 = new JButton("Red"),b2 = new JButton("Green"),
      b3 = new JButton("Blue"),b4 = new JButton("Yellow"),
      b5 = new JButton("Pink");
  CardLayout lo = new CardLayout();
  Container pane;
  public CardLayoutTest() {
    setTitle("CardLayout Test");
    pane = getContentPane();
    pane.setLayout(lo);
    setBounds(0,0,200,100);
    pane.add(b1,"1"); pane.add(b2,"2"); pane.add(b3,"3");
   pane.add(b4,"4"); pane.add(b5,"5");
    b1.addActionListener(this); b2.addActionListener(this);
    b3.addActionListener(this); b4.addActionListener(this);
    b5.addActionListener(this);
   }
```

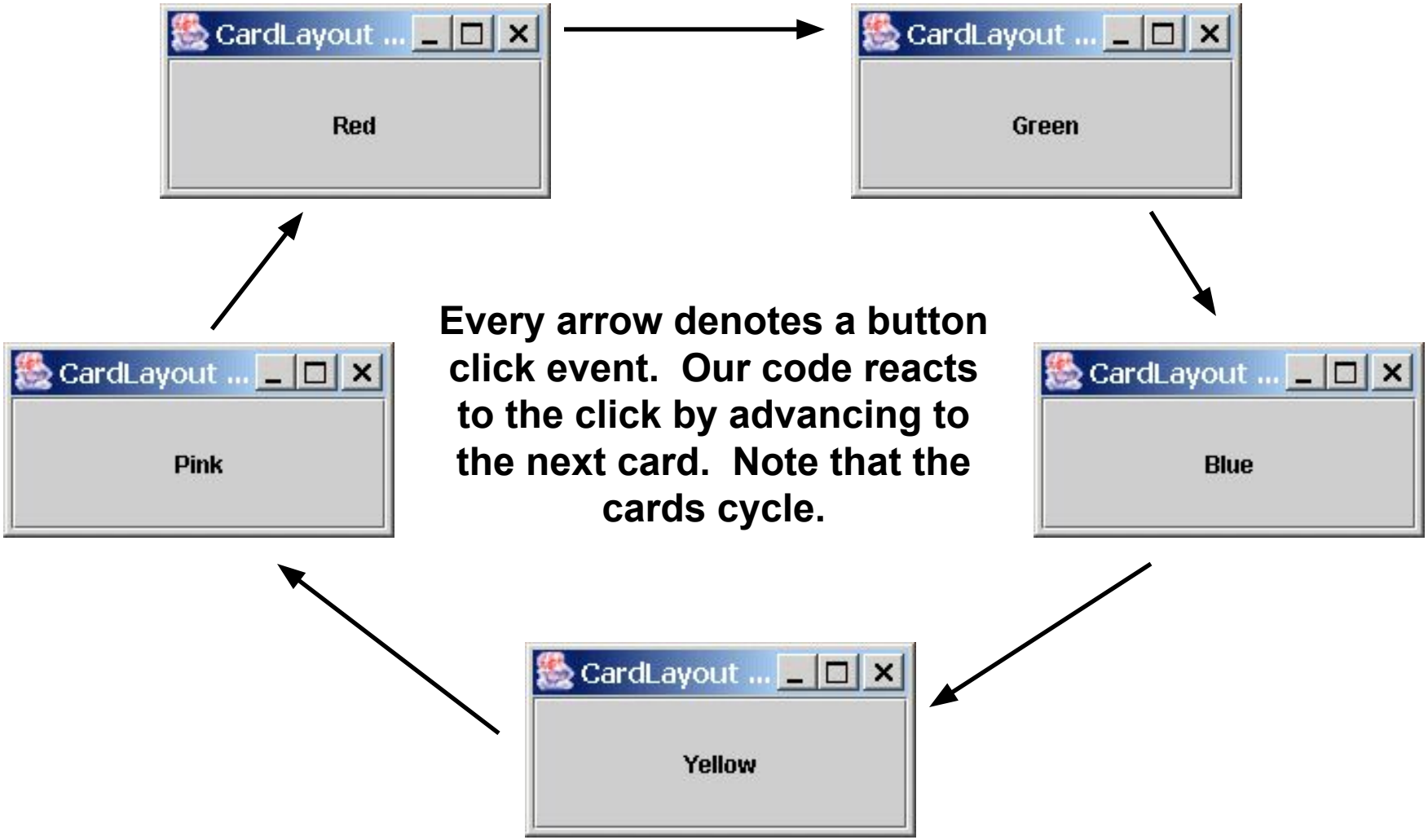
`CardLayout` Example (2 of 3)

```
// in the same file...
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == b1) lo.next(pane);
    else if (e.getSource() == b2) lo.next(pane);
    else if (e.getSource() == b3) lo.next(pane);
    else if (e.getSource() == b4) lo.next(pane);
    else if (e.getSource() == b5) lo.next(pane);
  }
  public static void main(String args[]) {
    JFrame f = new CardLayoutTest();
    f.setVisible(true);
  }
}
```



define the behavior when the user clicks a button: in this case, we advance to the next card

`CardLayout` Example (3 of 3)



Main Steps in GUI Programming

To make any graphic program work we must be able to create windows and add content to them.

To make this happen we must:

1. Import the `awt` or `swing` packages.
2. Set up a top-level container.
3. Fill the container with GUI components.
4. Install listeners for GUI Components.
5. Display the container.

Database Connectivity

Contents

- Overview
  - JDBC
  - Types of Drivers
  - API
- Connecting to Databases
- Executing Queries & Retrieving Results
- Advanced Topics
  - Prepared Statements
  - Connection Pooling

JDBC

Definition

- JDBC: Java Database Connectivity
  - It provides a standard library for Java programs to connect to a database and send it commands using SQL
  - It generalizes common database access functions into a set of common classes and methods
  - Abstracts vendor specific details into a code library making the connectivity to multiple databases transparent to user
- JDBC API Standardizes:
  - Way to establish connection to database
  - Approach to initiating queries
  - Method to create stored procedures
  - Data structure of the query result

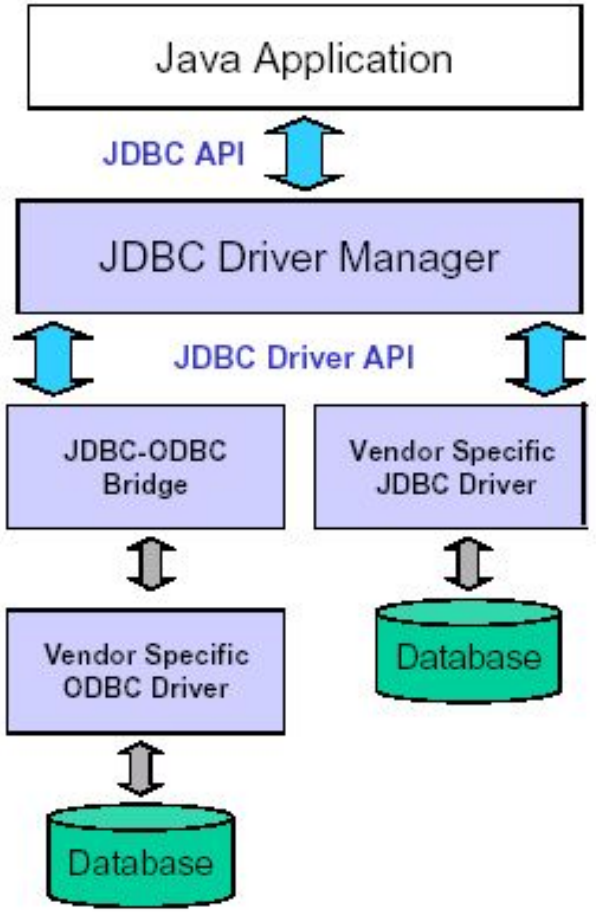
JDBC

API

- Two main packages java.sql and javax.sql
  - **Java.sql** contains all core classes required for accessing database (Part of Java 2 SDK, Standard Edition)
  - **Javax.sql** contains optional features in the JDBC 2.0 API (part of Java 2 SDK, Enterprise Edition)
- Javax.sql adds functionality for enterprise applications
  - DataSources
  - JNDI
  - Connection Pooling
  - Rowsets
  - Distributed Transactions

JDBC
Architecture

- JDBC Consists of two parts:
  - JDBC API, a purely Java-based API
  - JDBC Driver Manager, which communicates with vendor-specific drivers that perform the real communication with the database
- Translation to the vendor format occurs on the client
  - No changes needed to the server
  - Driver (translator) needed on client



JDBC

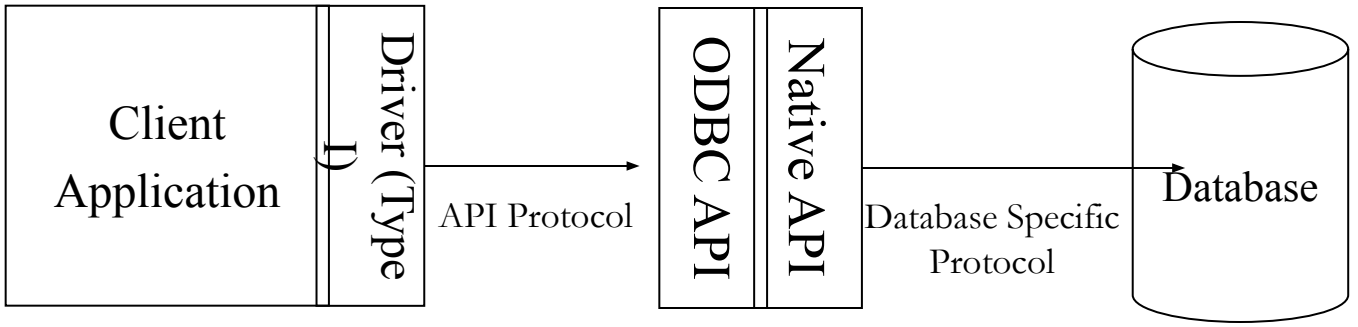
Drivers

- JDBC uses drivers to translate generalized JDBC calls into vendor-specific database calls
  - Drivers exist for most popular databases
  - Four Classes of JDBC drivers exist
    - Type I
    - Type II
    - Type III
    - Type IV

JDBC

Drivers (Type I)

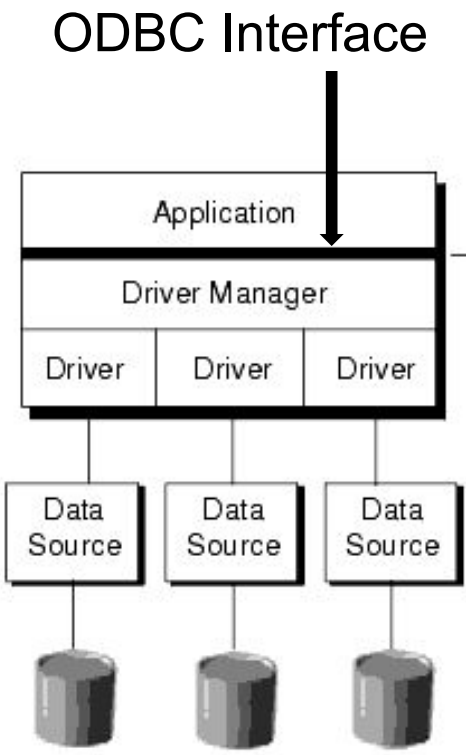
- Type I driver provides mapping between JDBC and access API of a database
  - The access API calls the native API of the database to establish communication
- A common Type I driver defines a JDBC to ODBC bridge
  - ODBC is the database connectivity for databases
  - JDBC driver translates JDBC calls to corresponding ODBC calls
  - Thus if ODBC driver exists for a database this bridge can be used to communicate with the database from a Java application
- Inefficient and narrow solution
  - Inefficient, because it goes through multiple layers
  - Narrow, since functionality of JDBC code limited to whatever ODBC supports



JDBC

Open Database Connectivity (ODBC)

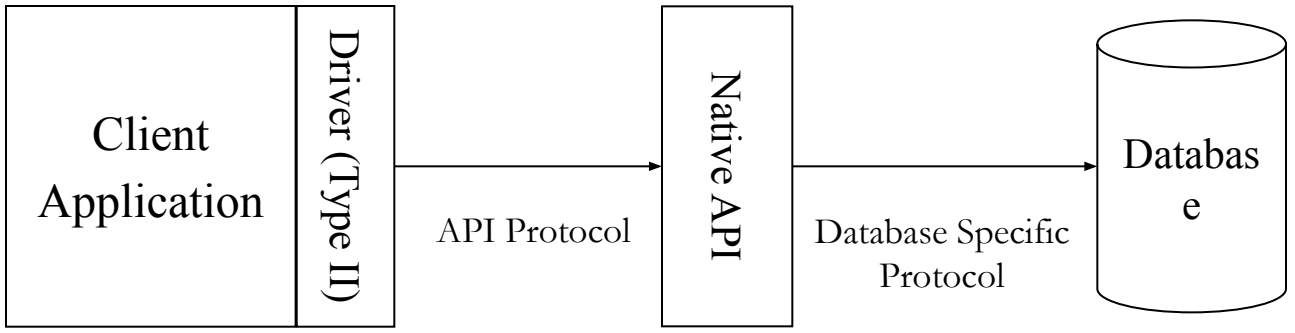
- A standard database access method developed by the SQL Access group in 1992.
  - The goal of ODBC is to make it possible to access any data from any application, regardless of which database management system (DBMS) is handling the data.
  - ODBC manages this by inserting a middle layer, called a database *driver* , between an application and the DBMS.
  - The purpose of this layer is to translate the application's data queries into commands that the DBMS understands.
  - For this to work, both the application and the DBMS must be *ODBC-compliant*, that is, the application must be capable of issuing ODBC commands and the DBMS must be capable of responding to them.



JDBC

Drivers (Type II)

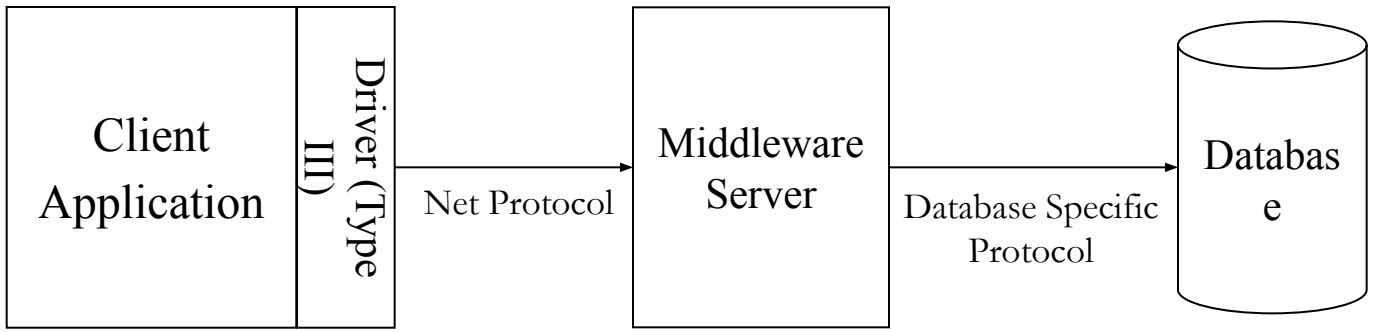
- Type II driver communicates directly with native API
  - Type II makes calls directly to the native API calls
  - More efficient since there is one less layer to contend with (i.e. no ODBC)
  - It is dependent on the existence of a native API for a database



JDBC

Drivers (Type III)

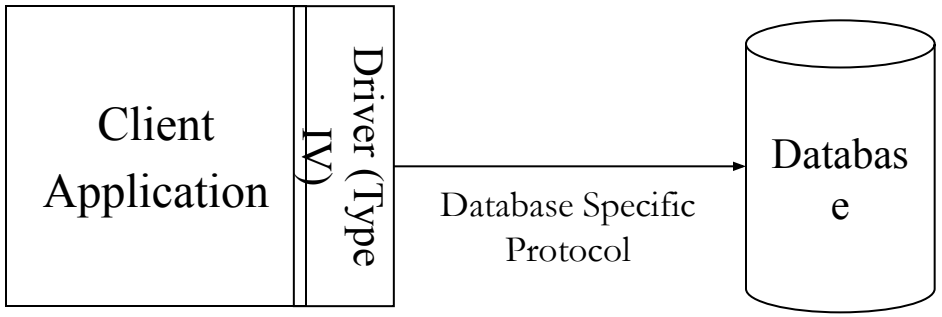
- Type III driver make calls to a middleware component running on another server
  - This communication uses a database independent net protocol
  - Middleware server then makes calls to the database using database-specific protocol
  - The program sends JDBC call through the JDBC driver to the middle tier
  - Middle-tier may use Type I or II JDBC driver to communicate with the database.



JDBC

Drivers (Type IV)

- Type IV driver is an all-Java driver that is also called a thin driver
  - It issues requests directly to the database using its native protocol
  - It can be used directly on platform with a JVM
  - Most efficient since requests only go through one layer
  - Simplest to deploy since no additional libraries or middle-ware

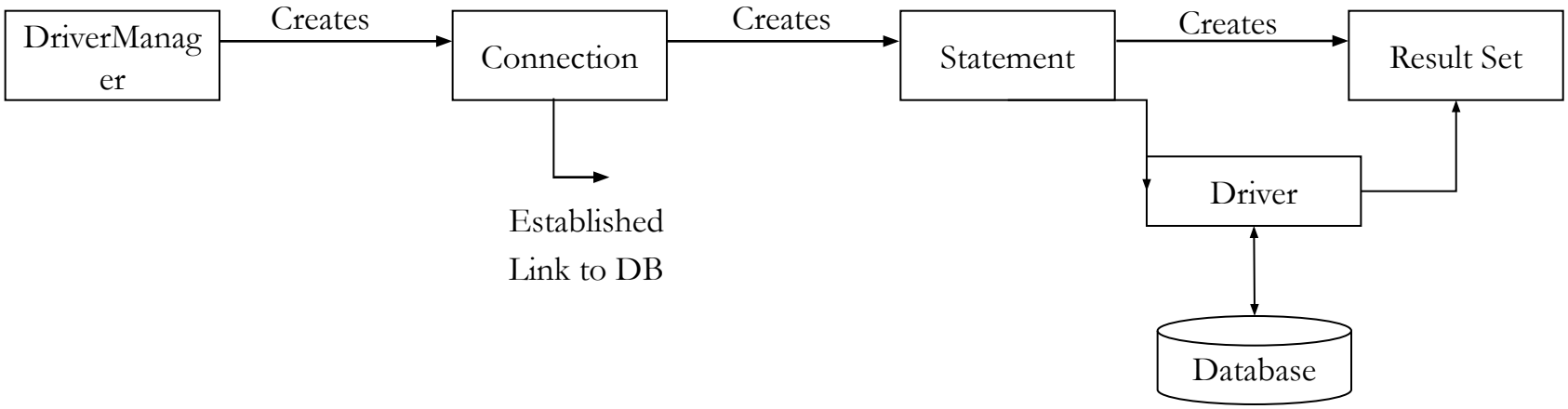


Connecting to Database

JDBC

Conceptual Components

- **Driver Manager:** Loads database drivers and manages connections between the application and the driver
- **Driver:** Translates API calls into operations for specific database
- **Connection:** Session between application and data source
- **Statement:** SQL statement to perform query or update
- **Metadata:** Information about returned data, database, & driver
- **Result Set:** Logical set of columns and rows of data returned by executing a statement



JDBC

Basic Steps

- Import the necessary classes
- Load the JDBC driver
- Identify the data source (Define the Connection URL)
- Establish the Connection
- Create a Statement Object
- Execute query string using Statement Object
- Retrieve data from the returned ResultSet Object
- Close ResultSet & Statement & Connection Object in order

JDBC

Driver Manager

- DriverManager provides a common access layer on top of different database drivers
  - Responsible for managing the JDBC drivers available to an application
  - Hands out connections to the client code
- Maintains reference to each driver
  - Checks with each driver to determine if it can handle the specified URL
  - The first suitable driver located is used to create a connection
- DriverManager class can not be instantiated
  - All methods of DriverManager are static
  - Constructor is private

JDBC Driver

Loading

- Required prior to communication with a database using JDBC
- It can be loaded
  - dynamically using Class.forName(String *drivername*)
  - System Automatically loads driver using jdbc.drivers system property
- An instance of driver must be registered with DriverManager class
- Each Driver class will typically
  - create an instance of itself and register itself with the driver manager
  - Register that instance automatically by calling RegisterDriver method of the DriverManager class
- Thus the code does not need to create an instance of the class or register explicitly using registerDriver(Driver) class

JDBC Driver

Loading: class.forName()

- Using forName(String) from java.lang.Class instructs the JVM to find, load and link the class identified by the String

```
e.g try {
        Class.forName("COM.cloudscape.core.JDBCDriver");
        } catch (ClassNotFoundException e) {
           System.out.println("Driver not found");
           e.printStackTrace();
        }
```

- At run time the class loader locates the driver class and loads it
  - All static initializations during this loading
  - Note that the name of the driver is a literal string thus the driver does not need to be present at compile time

JDBC Driver

Loading: System Property

- Put the driver name into the jdbc drivers System property
  - When a code calls one of the methods of the driver manager, the driver manager looks for the jdbc.drivers property
  - If the driver is found it is loaded by the Driver Manager
  - Multiple drivers can be specified in the property
  - Each driver is listed by full package specification and class name
  - a colon is used as the delimiter between the each driver

  e.g jdbc.drivers=com.pointbase.jdbc.jdbcUniversalDriver
- For specifying the property on the command line use:
  - java -Djdbc.drivers=com.pointbase.jdbc.jdbcUniversalDriver MyApp
- A list of drivers can also be provided using the Properties file
  - System.setProperty("jdbc.drivers", "COM.cloudscape.core.JDBCDriver");
  - DriverManager only loads classes once so the system property must be set prior to the any DriverManager method being called.

JDBC

URLs

- JDBC Urls provide a way to identify a database
- Syntax:

  <protocol>:<subprotocol>:<protocol>

  - Protocol: Protocol used to access database (jdbc here)
  - Subprotocol: Identifies the database driver
  - Subname: Name of the resource
- Example
  - Jdbc:cloudscape:Movies
  - Jdbc:odbc:Movies

Connection

Creation

- Required to communicate with a database via JDBC
- Three separate methods:

  public static Connection getConnection(String url)
  public static Connection getConnection(String url, Properties info)
  public static Connection getConnection(String url, String user, String password)

- Code Example (Access)

```
try {// Load the driver class
    System.out.println("Loading Class driver");
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    // Define the data source for the driver
    String sourceURL = "jdbc:odbc:music";
    // Create a connection through the DriverManager class
    System.out.println("Getting Connection");
    Connection databaseConnection = DriverManager.getConnection(sourceURL);
    }
 catch (ClassNotFoundException cnfe) {
        System.err.println(cnfe); }
catch (SQLException sqle) {
        System.err.println(sqle);}
```

Connection

Creation

- Code Example (Oracle)

```
try {
    Class.forName("oracle.jdbc.driver.OracleDriver");
    String sourceURL = "jdbc:oracle:thin:@delilah.bus.albany.edu:1521:databasename";
    String user = "goel";
    String password = "password";
    Connection databaseConnection=DriverManager.getConnection(sourceURL,user, password );
    System.out.println("Connected Connection"); }
catch (ClassNotFoundException cnfe) {
    System.err.println(cnfe); }
catch (SQLException sqle) {
    System.err.println(sqle);}
```

Connection

Closing

- Each machine has a limited number of connections (separate thread)
  - If connections are not closed the system will run out of resources and freeze
  - Syntax: public void close() throws SQLException

- Naïve Way:

```
try {
    Connection conn
    = DriverManager.getConnection(url);
     // Jdbc Code
    …
    } catch (SQLException sqle) {
      sqle.printStackTrace();
    }
    conn.close();
```

- SQL exception in the Jdbc code will prevent execution to reach conn.close()

- Correct way (Use the finally clause)

```
try{
Connection conn =
    Driver.Manager.getConnection(url);
    // JDBC Code
    } catch (SQLException sqle) {
      sqle.printStackTrace();
    } finally {
      try {
          conn.close();
      } catch (Exception e) {
          e.printStackTrace();
      }
}
```

Statement

Types

- Statements in JDBC abstract the SQL statements
- Primary interface to the tables in the database
- Used to create, retrieve, update & delete data (CRUD) from a table
  - Syntax: Statement statement = connection.createStatement();
- Three types of statements each reflecting a specific SQL statements
  - Statement
  - PreparedStatement
  - CallableStatement

Statement

Syntax

- Statement used to send SQL commands to the database
  - Case 1: ResultSet is non-scrollable and non-updateable

    ```
    public Statement createStatement() throws SQLException
    Statement statement = connection.createStatement();
    ```

  - Case 2: ResultSet is non-scrollable and/or non-updateable

    ```
    public Statement createStatement(int, int) throws SQLException
    Statement statement = connection.createStatement();
    ```

  - Case 3: ResultSet is non-scrollable and/or non-updateable and/or holdable

    ```
    public Statement createStatement(int, int, int) throws SQLException
    Statement statement = connection.createStatement();
    ```

- PreparedStatement

  ```
  public PreparedStatement prepareStatement(String sql) throws SQLException
  PreparedStatement pstatement = prepareStatement(sqlString);
  ```

- CallableStatement used to call stored procedures

  ```
  public CallableStatement prepareCall(String sql) throws SQLException
  ```

Statement

Release

- Statement can be used multiple times for sending a query
- It should be released when it is no longer required
  - Statement.close():
  - It releases the JDBC resources immediately instead of waiting for the statement to close automatically via garbage collection
- Garbage collection is done when an object is unreachable
  - An object is reachable if there is a chain of reference that reaches the object from some root reference
- Closing of the statement should be in the finally clause

```
try{
    Connection conn =
    Driver.Manager.getConnection(url);
    Statement stmt =
    conn.getStatement();
    // JDBC Code
    } catch (SQLException sqle) {
    sqle.printStackTrace();
    } finally {
      try {stmt.close();
            conn.close();
       } catch (Exception e) {
             e.printStackTrace();
       }
    }
```

JDBC
Logging

- DriverManager provides methods for managing output
  - DriverManagers debug output can be directed to a printwriter

    public static void setLogWriter(PrintWriter pw)
  - PrintWriter can be wrapped for any writer or OutputStream
  - Debug statements from the code can be sent to the log as well.

    public static void println(String s)
- Code

```
FileWriter fw = new FileWriter("mydebug.log");
PrintWriter pw = new PrintWriter(fw);
// Set the debug messages from Driver manager to pw
DriverManager.setLogWriter(pw);
// Send in your own debug messages to pw
DriverManager.println("The name of the database is " + databasename);
```

Querying the Database

Executing Queries

Methods

- Two primary methods in statement interface used for executing Queries
  - executeQuery Used to retrieve data from a database
  - executeUpdate: Used for creating, updating & deleting data
- executeQuery used to retrieve data from database
  - Primarily uses Select commands
- executeUpdate used for creating, updating & deleting data
  - SQL should contain Update, Insert or Delete commands
- Uset setQueryTimeout to specify a maximum delay to wait for results

Executing Queries

Data Definition Language (DDL)

- Data definition language queries use executeUpdate
- Syntax: int executeUpdate(String sqlString) throws SQLException
  - It returns an integer which is the number of rows updated
  - sqlString should be a valid String else an exception is thrown
- Example 1: Create a new table

```
Statement statement = connection.createStatement();
String sqlString =
"Create Table Catalog"
+ "(Title Varchar(256) Primary Key Not Null,"+
+ "LeadActor Varchar(256) Not Null, LeadActress Varchar(256) Not Null,"
+ "Type Varchar(20) Not Null, ReleaseDate Date Not NULL )";
Statement.executeUpdate(sqlString);
```

  - executeUpdate returns a zero since no row is updated

Executing Queries

DDL (Example)

- Example 2: Update table

```
Statement statement = connection.createStatement();
String sqlString =
 "Insert into Catalog"
+ "(Title, LeadActor, LeadActress, Type, ReleaseDate)"
+ "Values('Gone With The Wind', 'Clark Gable', 'Vivien Liegh',"
+ "'Romantic', '02/18/2003' "
Statement.executeUpdate(sqlString);
```

  - executeUpdate returns a 1 since one row is added

Executing Queries

Data Manipulation Language (DML)

- Data definition language queries use executeQuery
- Syntax

  ResultSet executeQuery(String sqlString) throws SQLException

  – It returns a ResultSet object which contains the results of the Query
- Example 1: Query a table

```
Statement statement = connection.createStatement();
String sqlString = "Select Catalog.Title, Catalog.LeadActor, Catalog.LeadActress," +
        "Catalog.Type, Catalog.ReleaseDate From Catalog";
ResultSet rs = statement.executeQuery(sqlString);
```

ResultSet

Definition

- ResultSet contains the results of the database query that are returned
- Allows the program to scroll through each row and read all columns of data
- ResultSet provides various access methods that take a column index or column name and returns the data
  - All methods may not be applicable to all resultsets depending on the method of creation of the statement.
- When the executeQuery method returns the ResultSet the cursor is placed before the first row of the data
  - Cursor refers to the set of rows returned by a query and is positioned on the row that is being accessed
  - To move the cursor to the first row of data next() method is invoked on the resultset
  - If the next row has a data the next() results true else it returns false and the cursor moves beyond the end of the data
- First column has index 1, not 0

ResultSet

- ResultSet contains the results of the database query that are returned
- Allows the program to scroll through each row and read all the columns of the data
- ResultSet provides various access methods that take a column index or column name and returns the data
  - All methods may not be applicable to all resultsets depending on the method of creation of the statement.
- When the executeQuery method returns the ResultSet the cursor is placed before the first row of the data
  - Cursor is a database term that refers to the set of rows returned by a query
  - The cursor is positioned on the row that is being accessed
  - First column has index 1, not 0
- Depending on the data numerous functions exist
  - getShort(), getInt(), getLong()
  - getFloat(), getDouble()
  - getClob(), getBlob(),
  - getDate(), getTime(), getArray(), getString()

ResultSet

- Examples:
  - Using column Index:

    Syntax:public String getString(int columnIndex) throws SQLException

    e.g. ResultSet rs = statement.executeQuery(sqlString);

    String data = rs.getString(1)
  - Using Column name

    public String getString(String columnName) throws SQLException

    e.g. ResultSet rs = statement.executeQuery(sqlString);

    String data = rs.getString(Name)
- The ResultSet can contain multiple records.
  - To view successive records next() function is used on the ResultSet
  - Example: while(rs.next()) {
  - System.out.println(rs.getString); }

Scrollable ResultSet

- ResultSet obtained from the statement created using the no argument constructor is:
  - Type forward only (non-scrollable)
  - Not updateable
- To create a scrollable ResultSet the following statement constructor is required
  - Statement createStatement(int resultSetType, int resultSetConcurrency)
- ResultSetType determines whether it is scrollable. It can have the following values:
  - ResultSet.TYPE_FORWARD_ONLY
  - ResultSet.TYPE_SCROLL_INSENSITIVE (Unaffected by changes to underlying database)
  - ResultSet.TYPE_SCROLL_SENSITIVE (Reflects changes to underlying database)
- ResultSetConcurrency determines whether data is updateable. Its possible values are
  - CONCUR_READ_ONLY
  - CONCUR_UPDATEABLE
- Not all database drivers may support these functionalities

Scrollable ResultSet

- On a scrollable ResultSet the following commands can be used
  - boolean next(), boolean previous(), boolean first(), boolean last()
  - void afterLast(), void beforeFirst()
  - boolean isFirst(), boolean isLast(), boolean isBeforeFirst(), boolean isAfterLast()
- Example

RowSet

- ResultSets limitation is that it needs to stay connected to the data source
  - It is not serializable and can not transporting across the network
- RowSet is an interface which removes the limitation
  - It can be connected to a dataset like the ResultSet
  - It can also cache the query results and detach from the database
- RowSet is a collection of rows
- RowSet implements a custom reader for accessing any tabular data
  - Spreadsheets, Relational Tables, Files
- RowSet object can be serialized and hence sent across the network
- RowSet object can update rows while diconnected fro the data source
  - It can connect to the data source and update the data
- Three separate implementations of RowSet
  - CachedRowSet
  - JdbcRowSet
  - WebRowSet

RowSet

- RowSet is derived from the BaseRowSet
  - Has SetXXX(…) methods to supply necessary information for making connection and executing a query
- Once a RowSet gets populated by execution of a query or from some other data source its data can be manipulated or more data added
- Three separate implementations of RowSet exist
  - CachedRowSet: Disconnected from data source, scrollable & serilaizable
  - JdbcRowSet: Maintains connection to data source
  - WebRowSet: Extension of CachedRowSet that can produce representation of its contents in XML

MetaData

- Meta Data means data about data
- Two kinds of meta data in JDBC
  - Database Metadata: To look up information about the database (here)
  - ResultSet Metadata: To get the structure of data that is returned (later)
- Example
  - connection.getMetaData().getDatabaseProductName()
  - connection.getMetaData().getDatabaseProductVersion()
- Sample Code:

```
private void showInfo(String driver,String url,String user,String password,
  String table,PrintWriter out) {
    Class.forName(driver);
    Conntection con = DriverManager.getConnection(url, username, password);
    DatabaseMetaData dbMetaData = connection.getMetaData();
    String productName = dbMetaData.getDatabaseProductName();
    System.out.println("Database: " + productName);
    String productVersion = dbMetaData.getDatabaseProductVersion();
    System.out.println("Version: " + productVersion);
}
```

Source Code

Connecting to Microsoft Access

```
/**
 * The code allows a user to connect to the MS Access Database and
 * run queries on the database. A sample query execution is provided
 * in this code. This is developed to help the students get initially
 * connected to the database.
 *
 * @author Sanjay Goel
 * @company School of Business, University at Albany
 *
 * @version 1.0
 * @created April 01, 2002 - 9:05 AM
 *
 * Notes 1: Statement is an interface hence can not be instantiated
 * using new. Need to call createStatement method of connection class
 *
 * Notes 2: Use executeQuery for DML queries that return a resultset
 * e.g., SELECT and Use executeUpdate for DDL & DML which do not
 * return Result Set e.g. (Insert Update and Delete) & DDL (Create
 * Table, Drop Table, Alter Table)
 *
 * */

import java.sql.*;

public class ConnectAccess {

    /**
     * This is the main function which connects to the Access database
     * and runs a simple query
     *
     * @param String[] args - Command line arguments for the program
     * @return void
     * @exception none
     *
     */
    public static void main(String[] args) {

// Load the driver
                try {
                    // Load the driver class
                    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

                    // Define the data source for the driver
                    String sourceURL = "jdbc:odbc:music";

                    // Create a connection through the DriverManager class
                    Connection databaseConnection
                        = DriverManager.getConnection(sourceURL);
                    System.out.println("Connected Connection");

                    // Create Statement
                    Statement statement = databaseConnection.createStatement();
                    String queryString
                        = "SELECT recordingtitle, listprice FROM recordings";

                    // Execute Query
                    ResultSet results = statement.executeQuery(queryString);

                    // Print results
                    while (results.next()){
                        System.out.println(results.getString("recordingtitle") +
                                                "\t" + results.getFloat("listprice"));
                    }

                    // Close Connection
                    databaseConnection.close();
                }
                catch (ClassNotFoundException cnfe) {
                    System.err.println(cnfe);
                }
                catch (SQLException sqle) {
                    System.err.println(sqle);
                }
    }
}
```

Connecting to Oracle

```
/**
 * The code allows a user to connect to the ORACLE Database and run
 * queries on the database. A sample query execution is provided in
 * this code. This is developed to help the students get initially
 * connected to the database.
 *
 * @author Sanjay Goel
 * @company School of Business, University at Albany
 *
 * @version 1.0
 * @created April 01, 2002 - 9:05 AM
 *
 * Notes 1: Statement is an interface hence can not be instantiated
 * using new. Need to call createStatement method of connection class
 *
 * Notes 2: Use executeQuery for DML queries that return a resultset
 * e.g., SELECT and Use executeUpdate for DDL & DML which do not
 * return Result Set e.g. (Insert Update and Delete) & DDL (Create
 * Table, Drop Table, Alter Table)
 *
 * */

import java.sql.*;

public class ConnectOracle {

    /**
     * This is the main function which connects to the Oracle database
     * and executes a sample query
     *
     * @param String[] args - Command line arguments for the program
     * @return void
     * @exception none
     *
     */
    public static void main(String[] args) {

// Load the driver
                try {
                    // Load the driver class
                    Class.forName("oracle.jdbc.driver.OracleDriver");

                    // Define the data source for the driver
                    String sourceURL
                         = "jdbc:oracle:thin:@delilah.bus.albany.edu:1521:bodb01";

                    // Create a connection through the DriverManager class
                    String user = "goel";
                    String password = "goel";
                    Connection databaseConnection
                         = DriverManager.getConnection(sourceURL, user, password);
                    System.out.println("Connected to Oracle");

                    // Create a statement
                    Statement statement = databaseConnection.createStatement();

                    // Create a query String
                    String sqlString = "SELECT artistid, artistname FROM artistsandperformers";

                    // Close Connection
                    databaseConnection.close();
                }
                catch (ClassNotFoundException cnfe) {
                    System.err.println(cnfe);
                }
                catch (SQLException sqle) {
                    System.err.println(sqle);
                }
        }
}
```

Connecting to Cloudscape

```
/**
 * The code allows a user to connect to the Cloudscape Database and
 * run queries on the database. A sample query execution is provided
 * in this code. This is developed to help the students get initially
 * connected to the database.
 *
 * @author Sanjay Goel
 * @company School of Business, University at Albany
 *
 * @version 1.0
 * @created April 01, 2002 - 9:05 AM
 *
 * Notes 1: Statement is an interface hence can not be instantiated
 * using new. Need to call createStatement method of connection class
 *
 * Notes 2: Use executeQuery for DML queries that return a resultset
 * e.g., SELECT and Use executeUpdate for DDL & DML which do not
 * return Result Set e.g. (Insert Update and Delete) & DDL (Create
 * Table, Drop Table, Alter Table)
 *
 * */

import java.sql.*;

public class ConnectCloudscape {

    public static void main(String[] args) {

        // Load the driver
        try {
            // Load the driver class
            Class.forName("COM.cloudscape.core.JDBCDriver");

            // Define the data source for the driver
            String sourceURL = "jdbc:cloudscape:Wrox4370.db";

            // Create a connection through the DriverManager class
                Connection databaseConnection =
            DriverManager.getConnection(sourceURL);
                System.out.println("Connected Connection");

                // Create a statement
                Statement statement = databaseConnection.createStatement();

                // Create an SQL statement
                String sqlString = "SELECT artistid, artistname FROM artistsandperformers";

                // Run Query
                ResultSet results = statement.executeQuery(sqlString);

                // Print Results
                while(results.next()) {
                    System.out.println(results.getInt("artistid") + "\t" +
                                       results.getString("artistname"));
                }

                // Close Connection
                databaseConnection.close();
            }
            catch (ClassNotFoundException cnfe) {
                System.err.println(cnfe);
            }
            catch (SQLException sqle) {
                System.err.println(sqle);
            }
    }
}
```

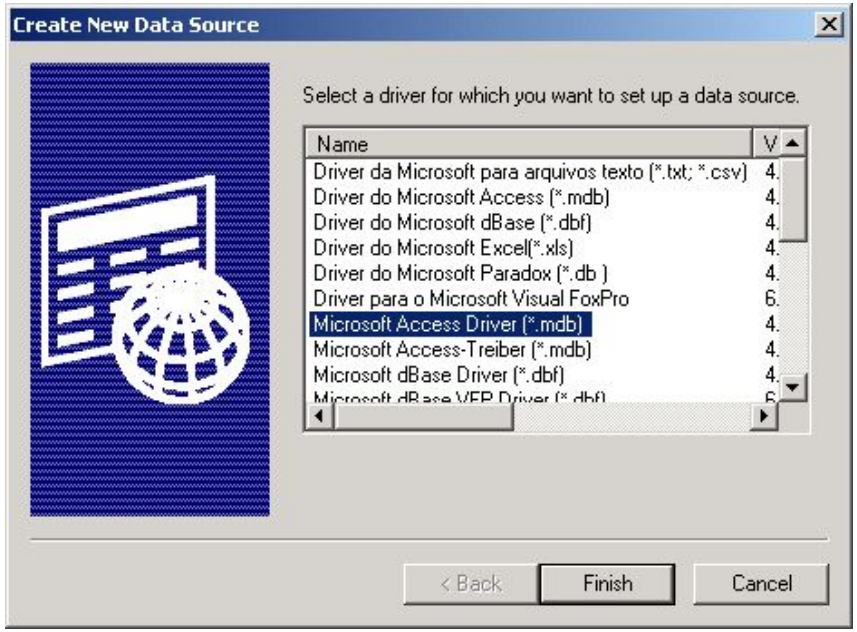
Prepared Statement

```
Import java.sql.*;                                                    // code from IVOr horton

public class AuthorDatabase {
  public static void main(String[] args) {
    try {
         String url = “jdbc:odbc:library”;
         String driver = “sun.jdbc.odbc.JdbcOdbcDriver”;
         String user = “goel”
         String password = “password”;
         // Load the Driver
         Class.forName(driver);
         Connection connection = DriverManager.getConnection();
         String sqlString = “UPDATE authors SET lastname = ? Authid = ?”;
         PreparedStatement ps = connection.prepareStatement(sqlString);
         // Sets first placeholder to Allamaraju
         ps.setString(1, “Allamaraju”);
        // Sets second placeholder to 212
        ps.setString(2, 212);
        // Executes the update
        int rowsUpdated = ps.executeUpdate();
        System.out.println(“Number of rows changed = “ + rowsUpdated);
        connection.close();
        }
      catch (ClassNotFoundException cnfe) {
        System.out.println(“Driver not found”);
        cnfe.printStackTrace();
       }
       catch (SQLException sqle) {
        System.out.println(“Bad SQL statement”);
        sqle.printStackTrace();
       }
```

Access Data Source

- Create a database
- Select DataSources (ODBC) from the control panel
  (Start Settings ControlPanel DataSources AdministrativeTools Data Sources)
- Select the System DSN tab
- On ODBC data source administrator click on add
- Select the database driver as Microsoft Access Driver



Access Data Source

- Fill the ODBC Microsoft Access Setup Form
  - Write Data Source Name (Name of the data source that you have in the program)
  - Add description of database
  - Click on select and browse the directory to pick a database file
  - Click on OK

Advanced Topics

JDBC – Data Types

JDBC Type	Java Type
BIT	boolean
TINYINT	byte
SMALLINT	short
INTEGER	int
BIGINT	long
REAL	float
FLOAT DOUBLE	double
BINARY VARBINARY LONGVARBINARY	byte[]
CHAR VARCHAR LONGVARCHAR	String

JDBC Type	Java Type
NUMERIC DECIMAL	BigDecimal
DATE	java.sql.Date
TIME TIMESTAMP	java.sql.Timestamp
CLOB	Clob*
BLOB	Blob*
ARRAY	Array*
DISTINCT	mapping of underlying type
STRUCT	Struct*
REF	Ref*
JAVA_OBJECT	underlying Java class

Prepared Statement

- PreparedStatement provides a means to create a reusable statement that is precompiled by the database
- Processing time of an SQL query consists of
  - Parsing the SQL string
  - Checking the Syntax
  - Checking the Semantics
- Parsing time is often longer than time required to run the query
- PreparedStatement is used to pass an SQL string to the database where it can be pre-processed for execution

Prepared Statement

- It has three main uses
  - Create parameterized statements such that data for parameters can be dynamically substituted
  - Create statements where data values may not be character strings
  - Precompiling SQL statements to avoid repeated compiling of the same SQL statement
- If parameters for the query are not set the driver returns an SQL Exception
- Only the no parameters versions of executeUpdate() and executeQuery() allowed with prepared statements.

Prepared Statement

- Example

```
// Creating a prepared Statement
String sqlString = "UPDATE authors SET lastname = ? Authid = ?";
PreparedStatement ps = connection.prepareStatement(sqlString);
ps.setString(1, "Allamaraju");     // Sets first placeholder to Allamaraju
ps.setString(2, 212);              // Sets second placeholder to 212
ps.executeUpdate();                // Executes the update
```

Callable Statements & Stored Procedures

- Stored Procedures
  - Are procedures that are stored in a database.
  - Consist of SQL statements as well as procedural language statements
  - May (or may not) take some arguments
  - May (or may not) return some values
- Advantages of Stored Procedures
  - Encapsulation & Reuse
  - Transaction Control
  - Standardization
- Disadvantages
  - Database specific (lose independence)
- Callable statements provide means of using stored procedures in the database

Callable Statements & Stored Procedures

- Stored Procedures must follow certain rules
  - Names of the stored procedures and parameters must be legal
  - Parameter types must be legal supported by database
  - Each parameter must have one of In, Out or Inout modes
- Example

  // Creating a stored procedure using SQL
  - CREATE PROC procProductsList AS SELECT * FROM Products;
  - CREATE PROC procProductsDeleteItem(inProductsID LONG) AS DELETE FROM Products WHERE ProductsID = inProductsID;“
  - CREATE PROC procProductsAddItem(inProductName VARCHAR(40), inSupplierID LONG, inCategoryID LONG) AS INSERT INTO Products (ProductName, SupplierID, CategoryID) Values (inProductName, inSupplierID, inCategoryID);"
  - CREATE PROC procProductsUpdateItem(inProductID LONG, inProductName VARCHAR(40)) AS UPDATE Products SET ProductName = inProductName WHERE ProductID = inProductID;"

  Usage: procProductsUpdateItem(1000, “My Music”)

  (Sets the name of the product with id 1000 to 16.99)

Thank You